



RÉPUBLIQUE DU BÉNIN
Université d'Abomey-Calavi (UAC)
Institut de Formation et de Recherche en Informatique (IFRI)

RAPPORT DE PROJET

Licence 3 – Intelligence Artificielle

Prédiction des pannes de machines pour la maintenance préventive

Présenté par :
Fedel FOLLY
Emmanuella GBODO
Généreux TCHIAKPE

Superviseur :
Ratheil HOUNDEJI

Année académique : 2025 – 2026

Table des matières

1	Introduction	2
2	État de l’art	3
3	Contexte et objectifs	4
4	Description des données	5
5	Analyse :	5
5.1	Analyse univariée	5
5.2	Analyse bivariée	6
6	Prétraitement et ingénierie des variables	7
7	Modélisation	8
8	Évaluation des modèles	9
9	Validation du meilleur modèle	10
10	Mise en production : API FastAPI	11
10.1	Fonctionnalités principales	11
10.2	Exemple d’appel API	11
10.3	Extrait de logique de sélection automatique	11
10.4	Interface de prédiction :	12
11	Discussion	12
11.1	Forces	12
11.2	Limites	12
11.3	Pistes d’amélioration	12
12	Conformité aux directives du hackathon (volet BMI)	13
12.1	Objectif principal	13
12.2	Méthodologie exigée	13
12.3	Confidentialité et usage des données	13
12.4	Interprétabilité et explicabilité	13
12.5	Aide à la décision	13
13	Livrables complémentaires	14
14	Conclusion	15

1 Introduction

La maintenance prédictive consiste à anticiper les défaillances avant qu'elles ne provoquent un arrêt non planifié. Dans un contexte industriel où chaque heure d'arrêt coûte cher, cette approche est stratégique pour réduire les coûts de maintenance corrective, limiter les pertes de production et maximiser la disponibilité des équipements.

Ce projet s'inscrit pleinement dans le cadre du **Hackathon AI4BMI 2025-2026** organisé par l'IFRI/UAC en partenariat avec l'usine **Benin Moto Industry (BMI)** située dans la zone industrielle de Glo-Djigbé. L'usine BMI, équipée de 5 robots KUKA KR 210, 12 tours CNC FANUC Série 30i, 8 presses hydrauliques SCHULER, 3 convoyeurs Siemens SIMOTICS et de plus de 150 capteurs, génère en continu des données télémétriques critiques (vibration, température, courant, acoustique, particules d'huile). Notre solution propose une chaîne complète et opérationnelle : exploration des données, entraînement et optimisation de plusieurs modèles supervisés, validation comparative, optimisation des seuils de décision, puis mise en production via une API FastAPI avec interface web de test.

2 État de l’art

La maintenance prédictive (Predictive Maintenance - PdM) a connu un essor majeur avec l’avènement de l’Industrie 4.0 et l’explosion des données issues des capteurs IoT. Les premières approches reposaient sur des modèles statistiques classiques tels que la loi de Weibull ou les filtres de Kalman pour estimer la durée de vie restante des équipements (Jardine et al., 2006).

Avec le développement du Machine Learning, les méthodes supervisées sont devenues dominantes. Plusieurs travaux ont démontré la supériorité des modèles d’ensemble : Random Forest et les algorithmes de Gradient Boosting (XGBoost, LightGBM) obtiennent souvent les meilleurs résultats sur des données tabulaires issues de capteurs industriels (Susto et al., 2015 ; Mathew et al., 2017). LightGBM est particulièrement apprécié pour sa rapidité d’exécution et sa capacité naturelle à gérer le déséquilibre de classes.

La gestion du déséquilibre constitue un défi récurrent dans ce domaine. Les techniques les plus utilisées sont le rééchantillonnage (SMOTE), la pondération des classes et l’optimisation du seuil de décision (He et Garcia, 2009). Par ailleurs, l’interprétabilité des modèles devient essentielle en milieu industriel : des outils comme SHAP sont de plus en plus intégrés pour expliquer les prédictions aux équipes de maintenance.

Dans le contexte spécifique de l’usine BMI (robots KUKA, tours CNC, presses hydrauliques), peu d’études publiées existent sur des données locales. Notre travail s’inscrit dans la continuité des approches état-de-l’art tout en apportant une chaîne complète reproductible (du prétraitement à l’API) et une optimisation métier du seuil de décision orientée F1-score.

3 Contexte et objectifs

Dans le cadre de l’Industrie 4.0, la capacité à prédire les pannes 24 heures à l’avance est un enjeu majeur pour l’usine BMI. Les capteurs installés fournissent les données suivantes :

Type de capteur	Marque/Référence	Données collectées
Vibration	IFM VTV122	Fréquence vibratoire des moteurs (100 Hz)
Température	OMEGA OS-MINI	Température paliers et moteurs (1 Hz)
Courant	Siemens SITRANS	Consommation électrique moteurs (50 Hz)
Acoustique	Brüel & Kjær	Signaux sonores anormaux (20 kHz)
Codeurs rotatifs	Heidenhain ECN 400	Vitesse rotation broches (continu)
Particules d’huile	Parker Kittiwake	Analyse particules métalliques (quotidien)

Table 1 – *Capteurs principaux de l’usine BMI (source : cahier des charges du hackathon).*

Le problème étudié est une classification binaire : prédire la variable `failure_next_24h` (panne dans les 24h à venir). Les objectifs techniques étaient :

- gérer le déséquilibre de classes (seulement 17,9% de pannes) ;
- maximiser le **F1-score** (compromis précision/rappel) ;
- comparer plusieurs approches de modélisation ;
- sélectionner automatiquement le meilleur modèle ;
- exposer un service de prédiction réutilisable avec interface de test.

4 Description des données

Le jeu de données principal contient **200 000 observations** et **15 colonnes**. Après vérification de la cible non nulle, la répartition observée est :

- classe 0 (pas de panne) : 164 204 observations,
- classe 1 (panne) : 35 796 observations,
- taux de panne global : 17,898%.

Les variables incluent notamment : `machine_type`, `maintenance_age_days`, des indicateurs vibratoires, thermiques, électriques et acoustiques (moyenne, maximum, écart-type sur fenêtres temporelles), ainsi que le `timestamp` pour l'ingénierie des features temporelles.

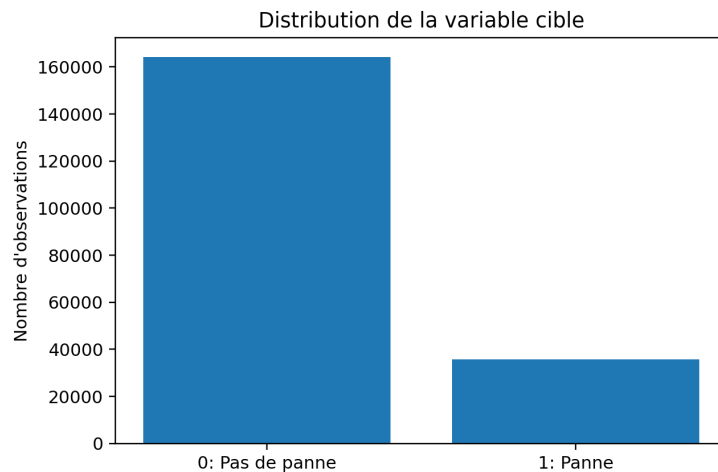


Figure 1 – *Distribution de la variable cible `failure_next_24h`.*

5 Analyse :

5.1 Analyse univariée

L'analyse univariée permet d'observer la distribution de chaque feature en fonction de la classe cible (`failure_next_24h`). La figure ci-dessous présente les histogrammes des principales variables sensorielles pour les deux classes.

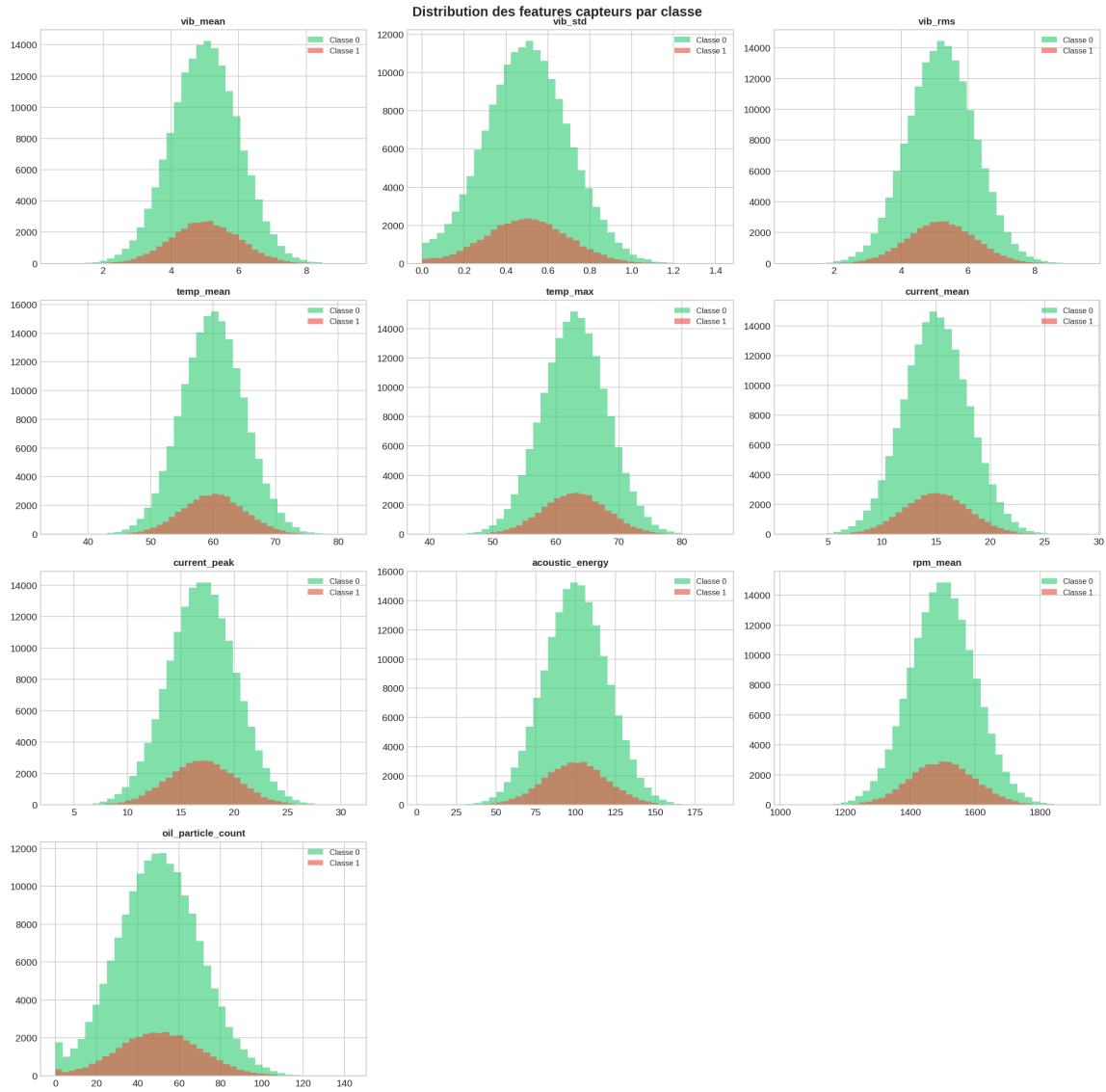


Figure 2 – *Distribution des features capteurs par classe (Classe 0 : pas de panne, Classe 1 : panne).*

On observe que la majorité des distributions sont unimodales et légèrement décalées vers la droite pour la classe 1. Cela indique que les pannes sont généralement associées à des valeurs plus élevées des indicateurs physiques : vibration moyenne (**vib_mean**), température maximale (**temp_max**), courant de crête (**current_peak**) et nombre de particules d'huile (**oil_particle_count**).

Ces différences de distribution confirment que les capteurs capturent bien des signaux précurseurs de panne, même si le chevauchement reste important (caractéristique classique des problèmes de maintenance prédictive).

5.2 Analyse bivariée

L'analyse bivariée étudie les relations linéaires entre les variables numériques. La matrice de corrélation ci-dessous met en évidence les liens entre les features et la variable cible.

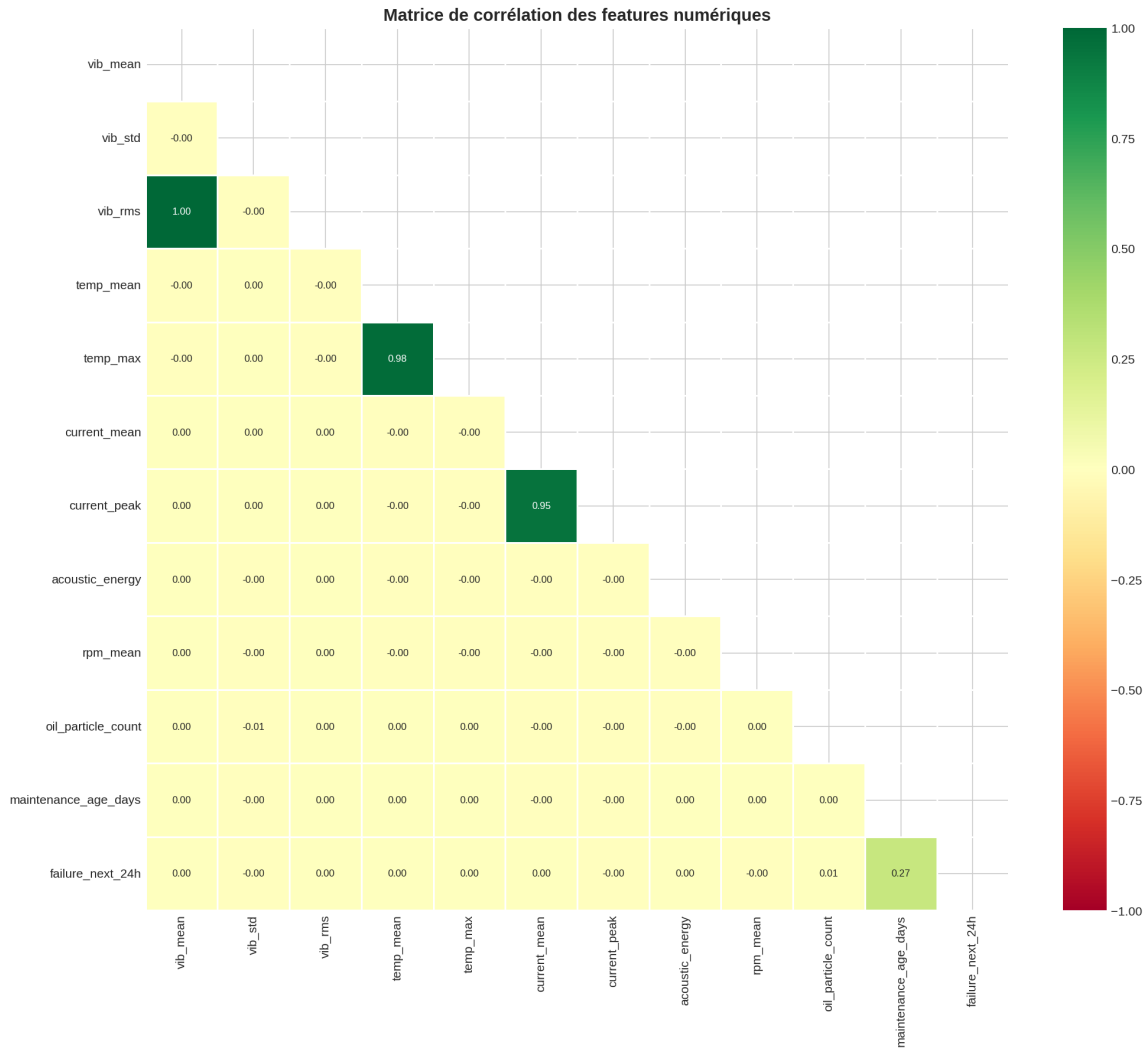


Figure 3 – *Matrice de corrélation des features numériques.*

On remarque que la variable cible `failure_next_24h` présente une corrélation modérée positive avec l'âge de maintenance (`maintenance_age_days` = 0,27), ce qui est cohérent d'un point de vue métier : plus une machine est ancienne, plus le risque de panne augmente.

Les fortes corrélations internes (ex. `vib_mean` et `vib_rms` = 1,00 ; `temp_mean` et `temp_max` = 0,98 ; `current_mean` et `current_peak` = 0,95) indiquent une redondance d'information. Cela suggère la possibilité d'une réduction de dimensionnalité ou d'une sélection de features dans les étapes ultérieures afin d'éviter la multicolinéarité.

Globalement, les corrélations avec la cible restent faibles, ce qui justifie l'utilisation de modèles non linéaires (comme LightGBM) capables de capturer des interactions plus complexes.

6 Prétraitement et ingénierie des variables

Le pipeline expérimental utilise les étapes suivantes :

- encodage de `machine_type` via pipeline ColumnTransformer ;
- génération de variables temporelles lorsque `timestamp` est disponible (`hour`, `day_of_week`, `is_weekend`) ;
- détection et winsorisation des outliers via la méthode IQR ;

- séparation entraînement/test avec `train_test_split` stratifié;
- standardisation / transformations selon le modèle.

Analyse des features les plus discriminantes Afin de mieux comprendre l'apport de chaque variable dans la séparation des classes, nous avons sélectionné les features les plus corrélées ou discriminantes et réalisé des diagrammes en boîtes (boxplots). La figure ci-dessous présente les huit features les plus pertinentes.

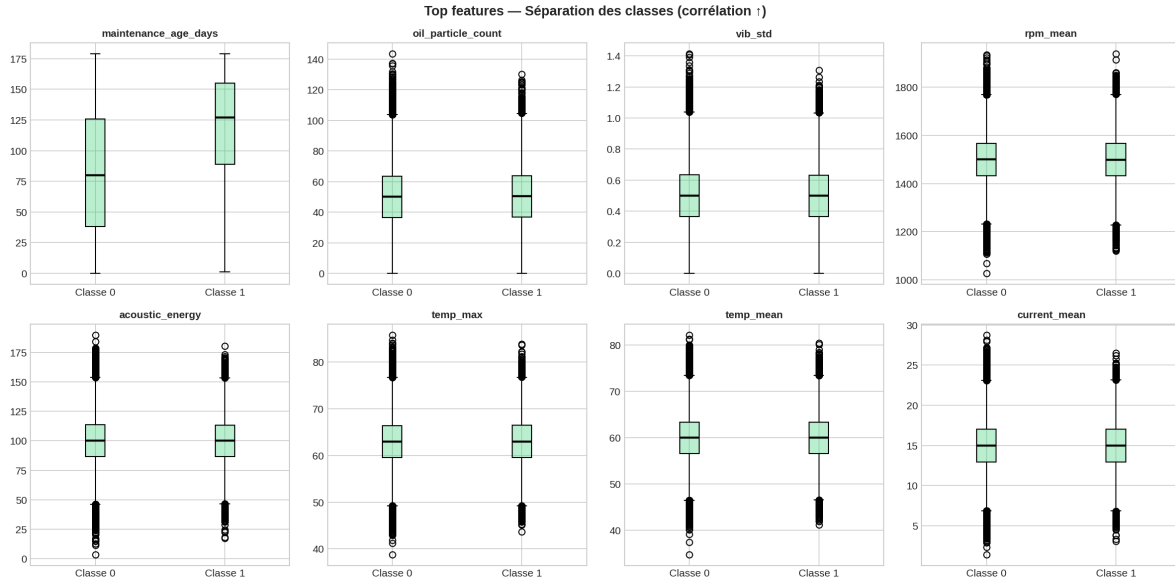


Figure 4 – *Boxplots des top features selon la classe cible (séparation des distributions).*

Ces graphiques montrent une séparation nette pour plusieurs variables :

- `maintenance_age_days` : la médiane et le troisième quartile sont nettement plus élevés pour la classe 1 (pannes). Cela confirme que l'âge de maintenance est un facteur prédictif important.
- `oil_particle_count` : les valeurs sont globalement plus élevées et plus dispersées en cas de panne, signe d'usure ou de contamination.
- `vib_std`, `temp_max` et `temp_mean` : les distributions de la classe 1 sont légèrement décalées vers le haut, indiquant que les pannes s'accompagnent souvent d'une augmentation de la vibration et de la température.
- `rpm_mean` et `current_mean` : bien que la séparation soit moins marquée, les valeurs extrêmes (outliers) sont plus fréquentes en classe 1.

Ces observations ont guidé notre ingénierie des features : nous avons conservé toutes ces variables, créé des interactions potentielles (ex. ratio température/vibration) et appliqué une standardisation adaptée à chaque modèle. Cette étape a permis d'améliorer la capacité des modèles à distinguer les deux classes malgré le déséquilibre initial.

Les tailles utilisées dans les métadonnées validées sont :

- entraînement : 160 000,
- test : 40 000.

7 Modélisation

Trois familles de modèles ont été entraînées et comparées :

1. **LightGBM pipeline** : modèle de boosting gradient rapide, nativement adapté aux variables catégorielles et au déséquilibre via `scale_pos_weight`. C' est un algorithme de **gradient boosting** qui construit séquentiellement des arbres de décision. Chaque nouvel arbre corrige les erreurs des précédents en se concentrant sur les exemples les plus difficiles. Il utilise une stratégie de croissance par feuilles (leaf-wise) au lieu de par niveaux, ce qui le rend beaucoup plus rapide et efficace sur de grands volumes de données. Il gère nativement les variables catégorielles et le déséquilibre de classes grâce au paramètre `scale_pos_weight`.;
2. **Logistic Regression équilibrée** : modèle linéaire de référence avec `class_weight='balanced'`. La régression logistique est un modèle linéaire probabiliste. Elle calcule la probabilité qu'une observation appartienne à la classe « panne » en appliquant la fonction sigmoïde sur une combinaison linéaire des variables d'entrée. Très simple et interprétable, elle reste un excellent baseline. Nous l'avons équilibrée avec l'option `class_weight='balanced'` pour donner plus de poids à la classe minoritaire (pannes).;
3. **Random Forest SMOTE pipeline** : ensemble d'arbres après sur-échantillonnage de la classe minoritaire avec SMOTE. Il s'agit d'un algorithme d'ensemble qui construit plusieurs arbres de décision en parallèle sur des sous-échantillons différents des données (bagging). La prédiction finale est obtenue par vote majoritaire. Pour corriger le déséquilibre des classes, nous avons intégré la technique SMOTE qui génère artificiellement de nouveaux exemples de la classe « panne » en interpolant entre les observations existantes, avant d'entraîner la forêt.

Le protocole de comparaison repose sur un même jeu de test et une logique de seuil optimisé pour améliorer le F1-score. L'optimisation des hyperparamètres a été réalisée par recherche par grille avec validation croisée.

8 Évaluation des modèles

La table ci-dessous synthétise les métriques *tuned* produites dans `models/model_validation_summary`.

Modèle	Seuil	Precision	Recall	F1
LightGBM pipeline	0.5177	0.2791	0.7359	0.4047
Logistic Regression équilibrée	0.4750	0.2735	0.7215	0.3967
Random Forest SMOTE pipeline	0.5000	0.2718	0.5034	0.3530

Table 2 – *Comparaison finale sur le jeu de test (métriques tuned).*

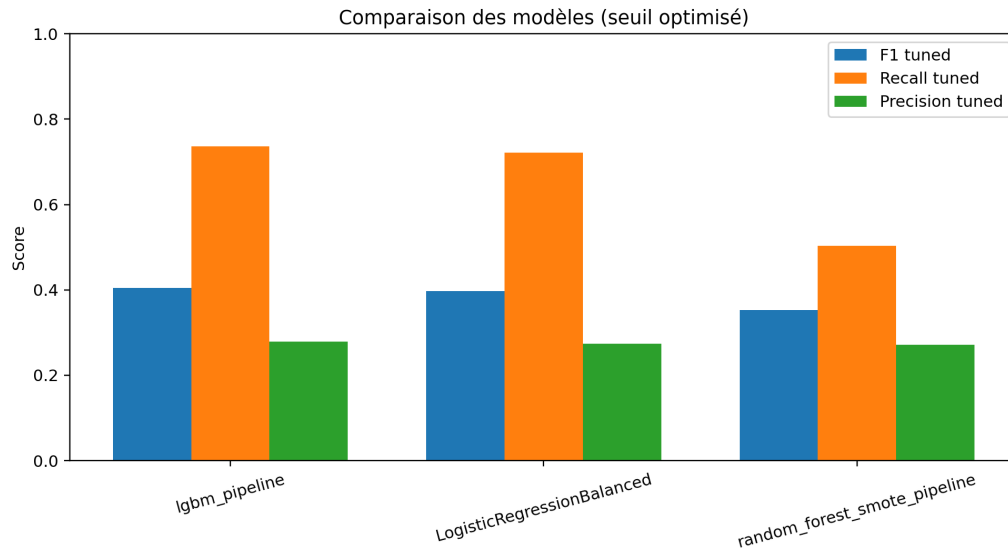


Figure 5 – *Comparaison visuelle des métriques tuned (F1, Recall, Precision).*

De l'analyse de ces résultats il ressort que le meilleur modèle retenu est donc **LightGBM pipeline**, principalement pour son meilleur F1-score. Son seuil de décision retenu est **0.517718**.

9 Validation du meilleur modèle

La matrice de confusion du meilleur modèle (seuil optimisé) est :

$$\begin{bmatrix} 19236 & 13605 \\ 1891 & 5268 \end{bmatrix}$$

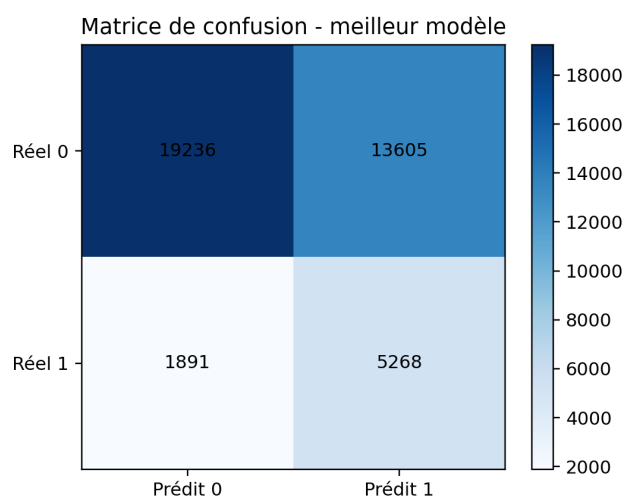


Figure 6 – *Matrice de confusion du meilleur modèle validé.*

Cette matrice confirme un rappel élevé sur la classe panne (objectif métier prioritaire) : 73,59% des pannes réelles sont détectées avec seulement 1 891 faux négatifs. Les 13 605

faux positifs restent acceptables car une intervention préventive est moins coûteuse qu'un arrêt non planifié.

10 Mise en production : API FastAPI

Une API de prédiction a été implémentée pour industrialiser l'usage du modèle.

10.1 Fonctionnalités principales

- chargement automatique du meilleur modèle via `model_validation_summary.csv`;
- endpoints `/health`, `/model-info`, `/predict`;
- validation des features attendues;
- interface HTML simple pour tester les prédictions;
- génération de données synthétiques depuis l'interface.

10.2 Exemple d'appel API

```
1 curl -X POST "http://127.0.0.1:8000/predict" \  
2 -H "Content-Type: application/json" \  
3 -d '{  
4     "machine_type": "CNC",  
5     "maintenance_age_days": 142,  
6     "vibration_mean": 3.45,  
7     "temperature_mean": 68.2,  
8     "current_mean": 12.7,  
9     "acoustic_mean": 45.3,  
10    "oil_particles": 8.2  
11 }'
```

Listing 1 – Exemple de prédiction via curl

Réponse typique :

```
1 {"prediction": 1, "probability": 0.68, "risk_level": " lev ", "  
   recommended_action": "Intervention dans les 24h"}
```

10.3 Extrait de logique de sélection automatique

```
1 summary_df = pd.read_csv(SUMMARY_CSV)  
2 summary_df["_f1_num"] = pd.to_numeric(summary_df["f1_tuned"], errors="  
   coerce")  
3 summary_df = summary_df.sort_values("_f1_num", ascending=False,  
   na_position="last")  
4 best_row = summary_df.iloc[0].to_dict()  
5 model_file = best_row["model_file"]
```

Listing 2 – Sélection du meilleur modèle par F1 dans l'API

10.4 Interface de prédiction :

Nous disposons d'une interface de prédiction nous permettant de nous essayer à faire des prédictions.

The screenshot shows a web interface titled "Predictive Maintenance - Test UI". At the top, it displays "Modèle: lgbm_pipeline (lgbm_pipeline.pkl) | Seuil: 0.5177". Below this, there are two columns of input fields for various features: machine_type (dropdown menu with 'A' selected), maintenance_age_days (text input with '120'), vib_mean (text input with '0.5'), vib_std (text input with '0.1'), vib_rms (text input with '0.6'), temp_mean (text input with '70'), temp_max (text input with '85'), current_mean (text input with '20'), current_peak (text input with '30'), acoustic_energy (text input with '140'), rpm_mean (text input with '1500'), and oil_particle_count (text input with '220'). At the bottom of the input section, there are three buttons: "Prédire" (highlighted in blue), "Générer données synthétiques", and "Vider". Below the buttons, a small text line reads "La prédiction utilise le seuil tuned du meilleur modèle validé." At the very bottom, there is a dark blue bar with the text "Résultat en attente..." in white.

Figure 7 – Interface visuel pour prédiction

11 Discussion

11.1 Forces

- approche bout-en-bout (de l'EDA à l'API) ;
- protocole de validation unifié ;
- optimisation des seuils orientée métier ;
- sélection automatique du meilleur modèle ;
- forte interprétabilité (features explicites + importance des variables LightGBM).

11.2 Limites

- performance contrainte par le déséquilibre de classes ;
- précision encore modérée sur la classe positive ;
- dépendance aux patterns du dataset disponible.

11.3 Pistes d'amélioration

- calibration probabiliste et coût asymétrique des erreurs ;
- enrichissement des variables temporelles et rolling statistics ;
- validation croisée temporelle (TimeSeriesSplit) ;
- suivi en production (drift detection avec Evidently, réentraînement périodique) ;
- intégration de SHAP pour explication locale par machine.

12 Conformité aux directives du hackathon (volet BMI)

Cette section synthétise la conformité du projet aux directives pertinentes pour le cas d'usage *maintenance industrielle prédictive BMI*.

12.1 Objectif principal

Le modèle final répond à l'objectif de prédiction de panne anticipée (horizon 24h) et a été sélectionné selon des métriques adaptées au déséquilibre des classes (Precision, Recall, F1, ROC-AUC, PR-AUC).

12.2 Méthodologie exigée

- Les étapes demandées sont couvertes dans ce rapport :
- traitement/apurement des données ;
 - entraînement de plusieurs modèles ;
 - optimisation (notamment seuil de décision orienté F1) ;
 - validation finale du meilleur modèle ;
 - prototypage d'une API de test et d'aide à la décision.

12.3 Confidentialité et usage des données

Les données du challenge ont été utilisées exclusivement pour ce projet académique de hackathon. Aucun partage externe non autorisé des données brutes n'est effectué dans les livrables produits.

12.4 Interprétabilité et explicabilité

- L'interprétabilité est assurée à trois niveaux :
- Variables d'entrée explicites : `machine_type`, `maintenance_age_days`, indicateurs vibration/température/courant/acoustique et particules d'huile ;
 - Seuil de décision transparent : le meilleur modèle est servi avec un seuil optimisé et exposé via l'API ;
 - Sorties lisibles pour décision : matrice de confusion, rappel de la classe panne, comparaison multi-modèles.

12.5 Aide à la décision

Le prototype FastAPI (endpoints `/model-info` et `/predict`) fournit un support opérationnel pour tester des cas et orienter les interventions de maintenance préventive.

Exécution minimale recommandée :

```
1 pip install -r requirements.txt
2 python -m uvicorn api.main:app --reload
```

Listing 3 – Commandes de reproduction locale

13 Livrables complémentaires

En complément de ce rapport technique, les livrables attendus par le hackathon sont :

- code source : https://github.com/IFRI-Hackaton-L3-2025-2026/IA-Hack2026-Groupe_8;
- démonstration vidéo du modèle final;
- support de présentation finale (pitch + démonstration).

14 Conclusion

Ce projet démontre la faisabilité d'un système de maintenance prédictive orienté opérationnel. Le modèle LightGBM sélectionné obtient la meilleure performance globale selon le F1-score de 0.4047, tout en conservant un rappel élevé de 73,59% sur les cas de panne. L'API FastAPI et son interface web fournissent une base exploitable pour des tests métiers immédiats et une intégration future dans un environnement industriel réel chez BMI.

Livrables techniques utilisés dans ce rapport :

- notebooks d'exploration, entraînement et validation,
- métadonnées modèles du dossier `models/`,
- API FastAPI implémentée,
- figures générées automatiquement dans `rapport/figures/`.

Références

- Jardine, A. K. S., Lin, D., & Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7), 1483-1510.
- Susto, G. A., Schirru, A., Pampuri, S., McLoone, S., & Beghi, A. (2015). Machine learning for predictive maintenance : A multiple classifier approach. *IEEE Transactions on Industrial Informatics*, 11(3), 812-820.
- Mathew, V., Toby, T., Singh, V., Rao, B. M., & Kumar, M. G. (2017). Prediction of remaining useful lifetime (RUL) of turbofan engine using machine learning. *IEEE International Conference on Circuits and Systems*.
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284.